BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO NGHIÊN CỨU

TÌM HIỂU VỀ TRACE CACHE

Giảng viên hướng dẫn: ThS. Cấn Thị Phượng

Nhóm thực hiện: Nhóm 9

Lớp: 64.CNTT-2

KHÁNH HÒA – 2023

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NHA TRANG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO NGHIÊN CỨU

TÌM HIỂU VỀ TRACE CACHE

Giảng viên hướng dẫn: ThS. Cấn Thị Phượng

Nhóm thực hiện: Nhóm 9

Lớp: 64.CNTT-2

KHÁNH HÒA - 2023

MỤC LỤC

MỤC LỤC i

DANH MỤC HÌNH VẼ, ĐỒ THỊ iii

DANH MỤC CÁC KÝ HIỆU, TỪ VIẾT TẮT iv

LỜI MỞ ĐẦU v

CHƯƠNG 1 TRACE 6

1.1 Khái niệm 6

1.2 Đặc tính (Trace Characterist) 6

1.3 Cấu trúc (Trace structure) 7

1.4 Trace chaining 8

1.5 Trace mapping 12

CHƯƠNG 2 TRACE CACHE 14

2.1 Khái niệm 14

2.2 Phân loại 14

2.3 Lịch sử phát triển của Trace Cache 15

CHƯƠNG 3 THIẾT KẾ CỦA TRACE CACHE 16

CHƯƠNG 4 CƠ CHẾ CỦA TRACE CACHE (CƠ CHẾ TÌM NẠP-THE FETCH MECHANISM) 17

CHƯƠNG 5 TRACE CACHE MISS VÀ HIT LOGIC 18

5.1 Giới thiệu 18

5.2 Cache hit 18

5.3 Cache miss 18

5.4 Hit and miss logic ở Trace Cache 19

5.5 Tác dụng 19

5.6 Cách tối ưu hóa hiệu năng thông qua Trace Cache hit và miss logic.... 19

CHƯƠNG 6 MẶT LỢI VÀ MẶT HẠI CỦA TRACE CACHE 20

TÀI LIỆU THAM KHẢO XXIII

DANH MỤC HÌNH VẼ, ĐỒ THỊ

Hình 1.1 Trace mapping cho một I-cache được sắp xếp trực tiếp.....13

Hình 1.2 Xác định kích thước của CFA từ tần số và kích thước thực hiện của các vết phổ biến nhất. 13

Hình 2.1 Sơ đồ Software Trace Cache 15

Hình 2.2 Sơ đồ Hardware Trace Cache.....15

Hình 3.1 Sơ đồ Trace Cache 16

Hình 4.1. Sơ đồ cơ chế tìm nạp ở Trace Cache 17

DANH MỤC CÁC KÝ HIỆU, TỪ VIẾT TẮT

CFA	Conflict Free Area
CISC	Complex Instruction Set Computer
FA	Fall-through Address
HTC	Hardware Trace Cache
IC	Instruction Cache
PC	Program Counter
RISC	Reduced Instruction Set Computer
STC	Software Trace Cache
TA	Target Address

LỜI MỞ ĐẦU

Trace Cache là một công nghệ quan trọng trong vi xử lý hiện đại, giúp tăng hiệu suất và giảm độ trễ của bộ xử lý. Nó có nhiều lợi ích và có thể được tùy chỉnh để phù hợp với các ứng dụng cụ thể.

Lý do chọn chủ đề này là vì tụi em muốn tìm hiểu và báo cáo về Cache, đặc biệt hơn là Trace Cache – 1 loại Cache. Bởi vì Trace Cache là 1 bộ nhớ đệm có tác dụng rất lớn đối với CPU, giúp mọi xử lí trên máy tính trở nên mượt mà hơn, mang lại trải nghiệm tốt cho người dùng.

Thông qua bản báo cáo này, ta có thể hiểu được Trace Cache là gì, nắm được khái niệm vết (Trace) để suy ra được các đặc tính của nó. Ngoài ra, bản báo cáo còn phải nêu ra được cơ chế và cấu trúc của Trace Cache. Đồng thời ta còn biết được mặt lợi và mặt trái của nó.

CHƯƠNG 1 TRACE

1.1 Khái niệm

Trace, còn được gọi là chuỗi lệnh động, là một mục trong bộ nhớ đệm theo dõi. Nó có thể được đặc trưng bởi số lệnh tối đa và khối cơ bản tối đa. Dấu vết có thể bắt đầu tại bất kỳ lệnh động nào. Nhiều dấu vết có thể có cùng một hướng dẫn bắt đầu, tức là, cùng một bộ đếm chương trình khởi động (PC) và các hướng dẫn từ các khối cơ bản khác nhau tùy theo kết quả nhánh.

1.2 Đặc tính (Trace Characterist)

1.2.1 Các đặc tính chính của trace

- Thông tin chi tiết về các hoạt động của chương trình: Trace cung cấp thông tin chi tiết về các hoạt động của chương trình, bao gồm các lệnh được thực thi, giá trị của các biến và các thông tin khác. Thông tin này giúp phân tích và sửa lỗi chương trình một cách hiệu quả.

- Khả năng ghi lại các hoạt động của chương trình trong thời gian thực: Trace có khả năng ghi lại các hoạt động của chương trình trong thời gian thực, giúp phát hiện lỗi nhanh chóng và hiệu quả. Ví dụ, nếu chương trình của bạn bị treo hoặc không phản hồi, trace có thể giúp bạn xác định nguyên nhân của vấn đề này.

- Khả năng phân tích và sửa lỗi chương trình một cách hiệu quả: Trace cung cấp thông tin chi tiết về các hoạt động của chương trình, giúp phân tích và sửa lỗi chương trình một cách hiệu quả. Ví dụ, nếu chương trình của bạn bị lỗi, trace có thể giúp bạn xác định vị trí của lỗi và sửa chữa nó một cách nhanh chóng.

- Sử dụng được với nhiều ngôn ngữ lập trình khác nhau: Trace có thể được sử dụng với nhiều ngôn ngữ lập trình khác nhau, giúp phát hiện lỗi và sửa chữa chương trình một cách hiệu quả.

1.2.2 Các loại trace phổ biến

- Trace hàm: Trace hàm giúp ghi lại các hoạt động của một hàm trong chương trình, bao gồm các tham số đầu vào, giá trị trả về và các hoạt động bên trong hàm. Thông tin này giúp phân tích và sửa lỗi hàm một cách hiệu quả.

- Trace câu lệnh: Trace câu lệnh giúp ghi lại các hoạt động của một câu lệnh trong chương trình, bao gồm các giá trị của biến và các hoạt động khác. Thông tin này giúp phân tích và sửa lỗi câu lệnh một cách hiệu quả.

- Trace biến: Trace biến giúp ghi lại các thay đổi của một biến trong chương trình, bao gồm giá trị của biến và thời gian thay đổi. Thông tin này giúp phân tích và sửa lỗi biến một cách hiệu quả.

- Trace mạng: Trace mạng giúp ghi lại các hoạt động mạng của chương trình, bao gồm các yêu cầu và phản hồi HTTP. Thông tin này giúp phân tích và sửa lỗi mạng một cách hiệu quả.

- Trace thời gian: Trace thời gian giúp ghi lại thời gian thực hiện của các hoạt động trong chương trình, bao gồm thời gian bắt đầu và kết thúc của mỗi hoạt động. Thông tin này giúp phân tích và tối ưu hóa chương trình một cách hiệu quả.

1.2.3 Các công cụ và phần mềm hỗ trợ trace, bao gồm các tính năng và ưu điểm của chúng.

Các công cụ và phần mềm hỗ trợ trace có nhiều tính năng và ưu điểm khác nhau, tùy thuộc vào mục đích sử dụng của từng công cụ. Dưới đây là một số ví dụ về các công cụ và phần mềm hỗ trợ trace:

- PyCharm: Là một IDE Python phổ biến, PyCharm cung cấp tính năng trace để giúp phân tích và sửa lỗi chương trình. PyCharm cũng hỗ trợ trace hàm, trace câu lệnh và trace biến.

- Visual Studio Code: Là một IDE đa nền tảng, Visual Studio Code cung cấp tính năng trace để giúp phân tích và sửa lỗi chương trình. Visual Studio Code cũng hỗ trợ trace hàm, trace câu lệnh và trace biến.

- Pdb: Là một công cụ trace Python tích hợp sẵn, Pdb cung cấp các tính năng trace hàm, trace câu lệnh và trace biến. Pdb cũng hỗ trợ các lệnh gỡ rối khác như bước từng dòng và xem giá trị biến.

- PySnooper: Là một công cụ trace Python đơn giản và dễ sử dụng, PySnooper cung cấp các tính năng trace hàm, trace câu lệnh và trace biến. PySnooper cũng hỗ trợ các tính năng như ghi log và định dạng đầu ra. Các công cụ và phần mềm hỗ trợ trace giúp phân tích và sửa lỗi chương trình một cách hiệu quả hơn.

Tùy thuộc vào nhu cầu sử dụng và kinh nghiệm của người dùng, các công cụ và phần mềm này có thể được lựa chọn để sử dụng trong quá trình phát triển chương trình.

1.3 Cấu trúc (Trace structure)

Trace thường chấm dứt khi xảy ra một trong các trường hợp sau:

- Trace đã được lấp đầy với số lượng hướng dẫn tối đa cho phép

- Trace có khối cơ bản tối đa cho phép

- Hướng dẫn trả lại
- Nhánh gián tiếp
- Cuộc gọi hệ thống

Một Trace duy nhất sẽ có thông tin sau:

- Khởi động PC - PC của lệnh đầu tiên trong dấu vết
- Cờ nhánh - (khối cơ bản tối đa -1) dự đoán nhánh
- Mặt nạ nhánh - số lượng nhánh trong dấu vết và liệu dấu vết có kết thúc trong nhánh hay không
- Dấu vết rơi qua - PC tiếp theo nếu lệnh cuối cùng không được thực hiện nhánh hoặc không phải là nhánh
- Mục tiêu theo dõi - địa chỉ của mục tiêu đã thực hiện của chi nhánh cuối cùng

1.4 Trace chaining

1.4.1 Khái niệm trace chaining

Trace chaining là một kỹ thuật được sử dụng trong vi xử lý để tăng hiệu suất đọc dữ liệu bằng cách cho phép Trace Cache lấy thành phần chung giữa nhiều luồng thực thi và kết hợp chúng thành một Trace Cache lớn hơn. Trace chaining sử dụng một cơ chế gọi là hybrid chaining để phủ nhận các trường hợp mà các trace không thể được kết hợp trong cùng một Trace Cache. Nó phù hợp với hệ thống đa luồng, nơi nhiều luồng có thể chia sẻ các trace được lưu trữ trong Trace Cache.

Khi trace chaining được áp dụng, các trace được lưu trữ trong Trace Cache không còn giới hạn trong một luồng cụ thể mà có thể được sử dụng bởi nhiều lõi thực thi khác nhau. Trong khi trace chaining có thể tăng hiệu suất đọc bộ nhớ, tuy nhiên nó cũng tiềm ẩn một số khó khăn. Một khi trace đã được lưu trữ trong cache, nó không thể được cập nhật khi có thay đổi trong câu lệnh đang thực thi, điều này có thể dẫn đến kết quả sai nếu các câu lệnh mới được thực hiện khác với những gì đã được lưu trữ trong trace.

1.4.2 Cấu tạo và đặc điểm của trace chaining

a. <u>Cấu tao:</u>

Trace chaining bao gồm Trace Cache và bộ nhớ cache. Trace Cache lưu trữ các trace được sử dụng thường xuyên của các luồng thực thi, trong khi đó bộ nhớ cache là nơi các dữ liệu được lưu trữ tạm thời để truy cập nhanh chóng.

b. <u>Đặc điểm của trace chaining bao gồm:</u>

- Tối ưu hoá việc sử dụng tài nguyên với sự cộng tác giữa Trace Cache và bộ nhớ cache.

- Cải thiện tính đa luồng, giúp các thread khác nhau sử dụng được các trace của nhau.

- Tuy nhiên, các trace phải được lưu trữ trong cache một cách liên tục để tránh pháp tuyến tính khi các trace không đủ liên tục với nhau.

1.4.3 Chức năng và vai trò của trace chaining

Trace chaining là một kỹ thuật được sử dụng trong vi xử lý để tăng hiệu suất đọc dữ liệu bằng cách cho phép Trace Cache lấy thành phần chung giữa nhiều luồng thực thi và kết hợp chúng thành một Trace Cache lớn hơn. Trace chaining giúp nâng cao hiệu suất của hệ thống máy tính bằng cách tận dụng tối đa tài nguyên bộ nhớ và tăng tốc độ thực thi các câu lệnh.

Trace chaining có vai trò quan trọng trong tối ưu hiệu suất của hệ thống máy tính. Cụ thể, trace chaining giúp tăng tốc độ thực thi và giảm thời gian truy cập bộ nhớ bằng cách tận dụng tối đa tài nguyên bộ nhớ và tái sử dụng các trace (chuỗi các câu lệnh thường xuyên được thực thi) được tạo ra bởi các luồng khác nhau của vi xử lý.

Điều này giúp tối ưu hoá việc sử dụng tài nguyên, cải thiện tính đa luồng bằng cách cho phép các thread sử dụng được các trace của nhau, đồng thời giảm phải tải dữ liệu từ bộ nhớ ngoài vào bộ nhớ trong, làm giảm thời gian chờ đợi của bộ vi xử lý khi truy cập bộ nhớ.

Vì vậy, trace chaining được áp dụng rất nhiều trong các vi xử lý hiện đại nhằm nâng cao hiệu suất và tối ưu hoá việc sử dụng tài nguyên của hệ thống.

1.4.4 Cách hoạt động của trace chaining

Trace Chaining là một kỹ thuật tối ưu hóa mã máy thường được sử dụng trong các trình thông dịch hoặc các JIT Compiler để cải thiện hiệu năng.

Trace Chaining hoạt động bằng cách xác định các đoạn mã máy (trace) được thực thi nhiều lần và kết nối chúng lại với nhau để tạo thành một đoạn mã "nóng" (hot path), giúp tối ưu hóa hiệu năng.

Các bước thực hiện Trace Chaining thường bao gồm:

1. Chạy chương trình và ghi lại dữ liệu về các trace được thực thi (trace logging).

2. Tạo ra một đoạn mã mới kết hợp các trace được thực hiện nhiều lần (trace stitching / chaining).

3. Thực thi đoạn mã mới để đảm bảo tính đúng đắn và hiệu năng.

Trace Chaining có thể được kết hợp với kỹ thuật khác như Inline Caching hoặc Guarding để giảm thiểu số lần kiểm tra điều kiện và tối ưu hóa hiệu suất.

Bên cạnh các ứng dụng trong các trình thông dịch hay JIT Compiler, Trace Chaining cũng được sử dụng trong các ứng dụng khác như trình duyệt hay các ứng dụng di động để tối ưu hóa hiệu suất ứng dụng.

1.4.5 Các kĩ thuật tối ưu cho việc thực thi các chương trình máy tính.

a. Direct branch chaining (Kết nối nhánh trực tiếp).

Nhánh trực tiếp, bất kể là điều kiện hay không điều kiện, thường dễ giải quyết bởi vì các địa chỉ đích của chúng không thay đổi trong quá trình thực thi chương trình. Nhánh trực tiếp không điều kiện có thể được loại bỏ đơn giản bằng cách thẳng hàng mã (nếu chúng không lưu trữ địa chỉ trả về, ví dụ như lệnh BR trong ISA Alpha) hoặc được thay đổi thành lệnh đặc biệt lưu địa chỉ trả về ban đầu (BSR). Nếu một nhánh điều kiện được thực thi tại thời điểm xây dựng trace, điều kiện của nó sẽ được đảo ngược để cho phép việc truy xuất liên tục. Nếu đích của nhánh tại điểm thoát trace (tức là đường dẫn không được theo sau tại thời điểm hình thành trace) không được tìm thấy trong bộ nhớ cache trace, một lệnh đặc biệt call-TM-if-condition-is-met được tạo ra cho nhánh điều kiện. Nếu đích của nhánh trở nên phổ biến và sau đó được đưa vào bộ nhớ cache trace, trình quản lý dịch (TM) trong hệ thống DSTC sẽ thay thế lệnh call-TM-if-condition-is-met bằng một lệnh nhánh điều kiện thông thường - một "patch" được thực hiện.

Tóm lại, Direct branch chaining là một kỹ thuật tối ưu hóa việc lấy lệnh trong kiến trúc máy tính. Khi một lệnh nhảy trực tiếp được thực hiện, nó sẽ nhảy đến một địa chỉ khác trong chương trình. Thay vì lấy lệnh tiếp theo từ địa chỉ hiện tại, direct branch chaining sẽ liên kết lệnh nhảy và các lệnh tiếp theo trong bộ nhớ để chúng có thể được lấy cùng nhau trong một hoạt động đọc duy nhất. Điều này giảm số lần truy cập bộ nhớ cần thiết để lấy các lệnh và cải thiện hiệu suất tổng thể của hệ thống.

Direct branch chaining có thể được sử dụng để tối ưu hóa việc lấy lệnh trong các kiến trúc máy tính khác nhau, bao gồm các kiến trúc RISC (Reduced Instruction Set Computer) và CISC (Complex Instruction Set Computer). Các hệ thống RISC thường sử dụng direct branch chaining để tối ưu hóa hiệu suất, trong khi các hệ thống CISC thường sử dụng các kỹ thuật tối ưu hóa khác như register-indirect jump chaining.

b. Register-indirect jump chaining (Kết nối nhảy gián tiếp thông qua thanh ghi).

Để tối ưu hóa các nhảy gián tiếp trong chương trình bằng cách sử dụng phương pháp dự đoán đích nhảy dựa trên phần mềm để giảm thiểu thời

gian tìm kiếm bảng cho mỗi lần nhảy gián tiếp. Trong các trình dịch tối ưu hóa động, một chuỗi các lệnh so sánh địa chỉ đích gián tiếp được lưu trữ trong một thanh ghi với địa chỉ đích được nhúng vào thời điểm dịch. Nếu có sự khớp, điều này cho thấy "dự đoán" là chính xác và lệnh đích được thực thi; nếu không, mã sẽ nhảy đến mã thay thế ở cuối trace, cuối cùng đến mã điều phối chung. Mặc dù dự đoán đích nhảy phần mềm thường đúng, nó vẫn tăng thêm thời gian xử lý cho chuỗi lệnh so sánh và nhảy. Nếu dự đoán thất bại, mã thay thế và mã điều phối vẫn phải được thực thi. Xác định đây là chi phí cao nhất và báo cáo rằng một lần tìm kiếm nhảy gián tiếp mất khoảng 15 lệnh, trong khi so sánh đích của một nhảy gián tiếp mất khoảng 6 lệnh trong ISA x86. Chaining register-indirect jump theo cách này cũng ảnh hưởng tiêu cực đến hành vi dự đoán nhánh vì nó thay thế một lệnh nhảy bằng một nhánh điều kiện và một nhảy gián tiếp khác ở cuối mã điều phối. Tỷ lệ dự đoán đích của nhảy gián tiếp cuối cùng này được dự đoán sẽ rất thấp vì tất cả các nhảy gián tiếp "dự đoán sai" đều dẫn đến cùng một mã điều phối và chỉ cần một mục BTB để cung cấp tất cả các địa chỉ đích. Một phương án thay thế khác là nhân bản mã điều phối sau mỗi nhảy gián tiếp qua thanh ghi, cho phép dự đoán đích riêng tư trong trường hợp dự đoán tại thời điểm xây dựng trace thất bại. Tùy chọn này đánh đổi kích thước trace, dẫn đến tăng áp lực I-cache, để cải thiện tỷ lệ dự đoán đích.

Tóm lại, register-indirect jump chaining là một kỹ thuật tối ưu hóa việc lấy lệnh trong kiến trúc máy tính. Khi một lệnh nhảy không trực tiếp được thực hiện, nó sẽ nhảy đến địa chỉ được lưu trữ trong một thanh ghi (register) thay vì một địa chỉ cụ thể trong chương trình. Thay vì lấy lệnh tiếp theo từ địa chỉ hiện tại, register-indirect jump chaining sẽ liên kết lệnh nhảy và các lệnh tiếp theo trong bộ nhớ để chúng có thể được lấy cùng nhau trong một hoạt động đọc duy nhất. Điều này giảm số lần truy cập bộ nhớ cần thiết để lấy các lệnh và cải thiện hiệu suất tổng thể của hệ thống.

Register-indirect jump chaining là một kỹ thuật tối ưu hóa thường được sử dụng trong các kiến trúc máy tính CISC (Complex Instruction Set Computer), trong khi trong các kiến trúc RISC (Reduced Instruction Set Computer) thì sử dụng direct branch chaining. Tuy nhiên, kỹ thuật register-indirect jump chaining cũng có thể được sử dụng trong các kiến trúc RISC trong một số trường hợp đặc biệt.

1.4.6 Thành tựu của trace chaining

Trace chaining đã đóng góp rất nhiều vào việc tăng tốc độ xử lý và hiệu suất của hệ thống máy tính.

Các thành tựu của trace chaining bao gồm:

- Tăng tốc độ thực thi chương trình: Trace chaining giúp giảm thời gian truy cập bộ nhớ bằng cách sử dụng tối đa các trace được tái sử dụng.

Điều này giúp tăng tốc độ thực thi chương trình và làm giảm thời gian đợi trong quá trình xử lý.

- Giảm phải tải dữ liệu từ bộ nhớ ngoài vào bộ nhớ trong: Trace chaining cho phép các thread sử dụng được các trace được tạo ra bởi các thread khác, do đó giảm phải tải dữ liệu từ bộ nhớ ngoài vào bộ nhớ trong, giúp giảm thời gian chờ đợi của bộ vi xử lý và tăng hiệu suất hệ thống.

- Tối ưu hoá việc sử dụng tài nguyên: Trace chaining giúp tối ưu hoá việc sử dụng tài nguyên bằng cách sử dụng lại các trace đã được tạo ra trước đó thay vì tạo mới các trace mới. Điều này giúp tối ưu hoá việc sử dụng bộ nhớ và tăng hiệu suất của hệ thống.

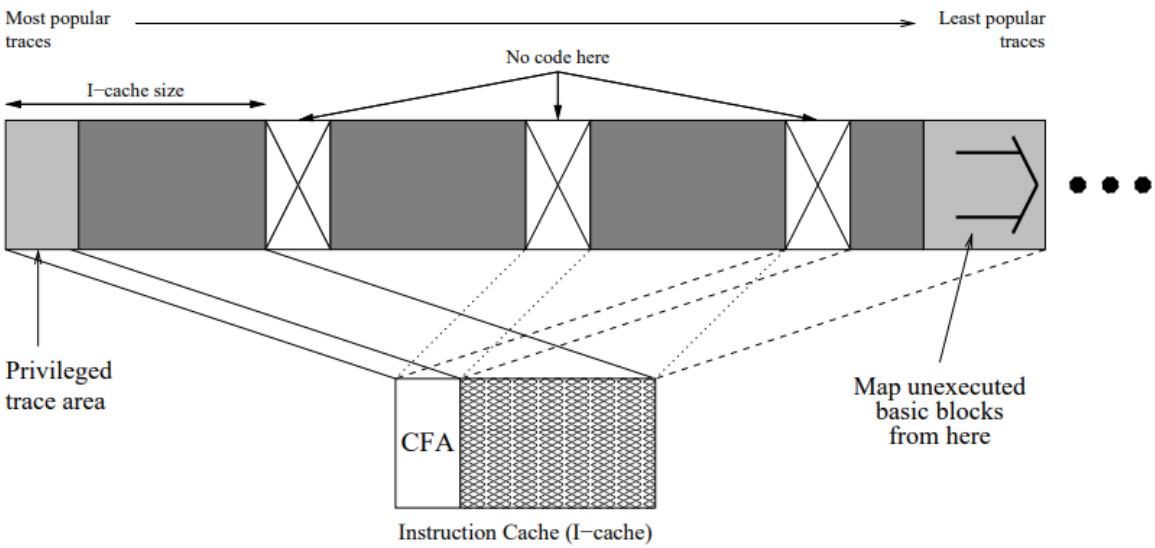
Những thành tựu này đã đóng góp rất lớn vào việc nâng cao hiệu suất và tăng tốc độ xử lý của hệ thống máy tính.

1.5 Trace mapping

Trace mapping được hiểu là sự sắp xếp của các Trace (vết). Như ta đã thấy trong hình 1.1, ta sắp xếp các vết đã nhận được theo thứ tự chúng được tạo: từ vết được thực hiện thường xuyên nhất đến vết được thực hiện ít nhất. Bằng cách này, Ta sắp xếp các vết phổ biến ngang nhau cạnh nhau, giảm xung đột giữa chúng. Ngoài ra, ta còn chia các vết trong instruction cache (I-cache) và để lại một khoảng trống ở đầu mỗi khối ngoại trừ khối đầu tiên (khối chứa các vết phổ biến nhất).

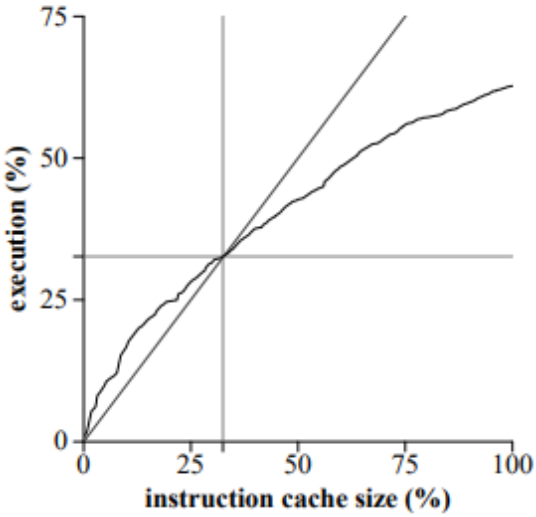
Tất cả các khoảng trống của mã sắp xếp tới cùng một vị trí trong instruction cache (I-cache), do đó không có mã nào khác sắp xếp tới cùng một vị trí với các vết phổ biến nhất, tạo ra một khu vực không xung đột (Conflict free area) (CFA) cho các vết này, giúp chúng hoàn toàn không bị can thiệp.

Kích thước của CFA là một trong những yếu tố quyết định nhất đến hiệu suất thu được khi sử dụng thuật toán sắp xếp này. Một CFA lớn hơn phù hợp với nhiều hơn các vết phổ biến nhất, bảo vệ chúng khỏi sự can thiệp, cái mà giúp giảm thiểu đi những cái xung đột bỏ lỡ trong các phân đoạn quan trọng nhất của mã. Tuy nhiên, nó để lại ít không gian hơn trong instruction cache (I-cache) cho các vết còn lại, làm tăng các lỗi xung đột giữa chúng. Cả hai yếu tố này đều cân bằng lẫn nhau và sau một kích thước nhất định, việc tăng thêm kích thước CFA thực sự sẽ làm giảm hiệu suất của instruction cache (I-cache).



Hình 1.1 Trace mapping cho một I-cache được sắp xếp trực tiếp

Dựa trên một nghiên cứu có sẵn, ta sẽ xác định kích thước CFA phù hợp mà không yêu cầu phương pháp thử và sai. Hình 1.2 cho thấy một ví dụ về cách ta xác định kích thước phù hợp cho CFA.



Hình 1.2 Xác định kích thước của CFA từ tần số và kích thước thực hiện của các vết phổ biến nhất.

Ta sẽ lấy từng vết phổ biến nhất. Sau đó, ta so sánh tỷ lệ phần trăm của tổng thời gian thực thi mà nó thu thập được so với phần trăm của bộ nhớ đệm lệnh mà nó yêu cầu. Nếu phần trăm thực thi cao hơn dung lượng chiếm dụng trong bộ nhớ đệm, ta sẽ đưa vết vào CFA. Sau đó, ta thêm vết tiếp theo và xem xét phần trăm thực thi mà chúng thực hiện cùng nhau và phần bộ nhớ cache mà chúng yêu cầu. Miễn là phần thực thi lớn hơn phần bộ instruction cache (I-cache) mà chúng yêu cầu, ta tiếp tục thêm dấu vết vào CFA để cố gắng cân bằng hai yếu tố. Trong ví dụ của Hình 1.2, thì ta sẽ

dành 32% I-cache cho CFA, vì nó thu thập chính xác 32% việc thực thi chương trình.

Phương pháp phỏng đoán này phụ thuộc vào tần suất thực thi của các vết được xây dựng và kích thước của I-cache. Đối với các bộ đệm nhỏ, kích thước của CFA cũng sẽ nhỏ hơn, trong khi các bộ đệm lớn hơn cho phép CFA lớn hơn. Các mã nhỏ hơn tập trung hầu hết quá trình thực thi của chúng trong một vài vết, sẽ gần như hoàn toàn phù hợp với CFA, trong khi các mã lớn có các cấu hình thực thi được thừa nhận sẽ ít hoặc không được sử dụng cho CFA

CHƯƠNG 2 TRACE CACHE

2.1 Khái niệm

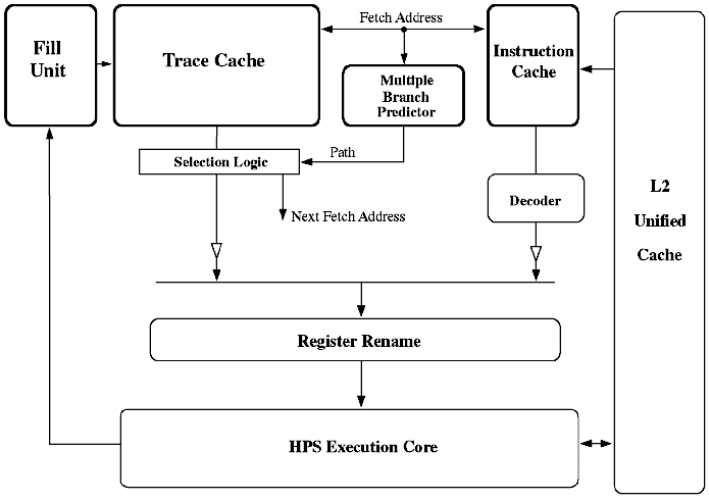
Trace Cache là một bộ nhớ cache được sử dụng trong các bộ vi xử lý để lưu trữ các chuỗi các lệnh liên tiếp được thực thi trong chương trình. Khi một chuỗi lệnh được thực thi, Trace Cache sẽ lưu trữ nó vào bộ nhớ cache để sử dụng lại trong tương lai nếu cần thiết. Trace Cache sử dụng một kỹ thuật gọi là "tracing" để xác định các chuỗi lệnh liên tiếp và lưu trữ chúng trong bộ nhớ cache. Khi một chuỗi lệnh mới được thực thi, Trace Cache sẽ kiểm tra xem nó có trùng với một chuỗi lệnh đã được lưu trữ trong cache hay không. Nếu có, nó sẽ sử dụng lại chuỗi lệnh đó từ cache thay vì truy cập bộ nhớ chính. Điều này giúp tăng tốc độ thực thi chương trình và giảm thời gian truy cập bộ nhớ chính.

Trace Cache là một cấu trúc bộ nhớ cache đặc biệt được sử dụng trong vi xử lý để tăng hiệu suất thực thi chương trình. Trace Cache lưu trữ các chuỗi lệnh liên tiếp (gọi là "trace"), thay vì lưu trữ các lệnh riêng lẻ. Một trace bao gồm các lệnh liên quan đến nhau, thường là một khối lệnh hoàn chỉnh của một hàm hoặc một vòng lặp. Khi CPU thực thi một trace, các lệnh trong trace được lấy ra từ Trace Cache và được thực thi mà không cần phải truy cập bộ nhớ chính. Điều này giúp giảm thời gian truy cập bộ nhớ và tăng tốc độ thực thi chương trình. Trace Cache thường được sử dụng trong các vi xử lý có kiến trúc RISC hoặc VLIW.

2.2 Phân loại

2.2.1 Software Trace Cache

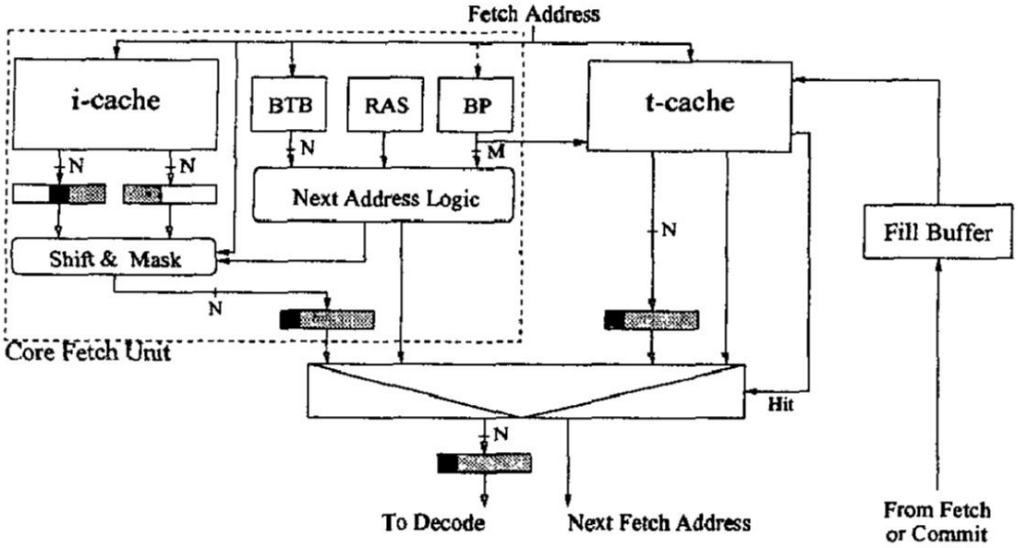
Thật ra Software Trace Cache hay còn gọi là STC , được hiểu là Trace Cache thông thường



Hình 2.1 Sơ đồ Software Trace Cache

2.2.2 Hardware Trace Cache

Là sự tương tác giữa kỹ thuật phần mềm và phần cứng nhắm mục tiêu tăng hướng dẫn tìm nạp băng thông.



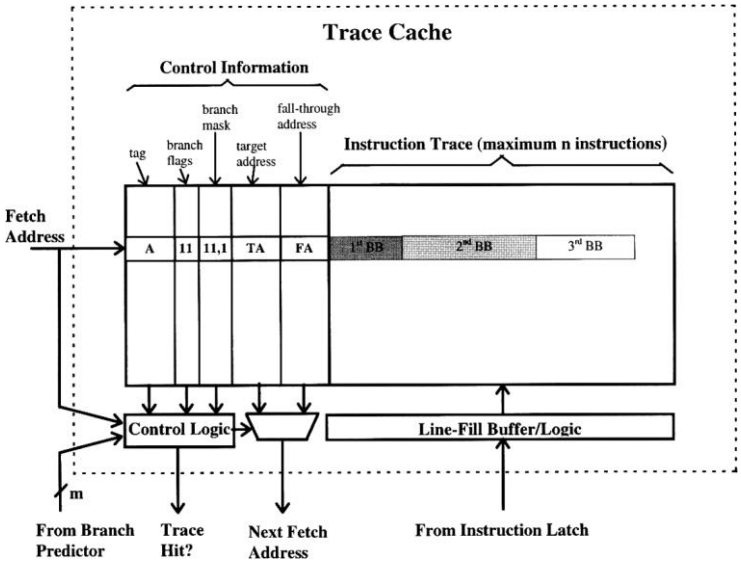
Hình 2.2 Sơ đồ Hardware Trace Cache

2.3 Lịch sử phát triển của Trace Cache

Trace Cache được giới thiệu lần đầu tiên vào năm 2000 bởi Intel trong bộ vi xử lý Pentium 4. Nó đã được sử dụng trong các bộ vi xử lý của Intel và AMD, và đã trở thành một phần quan trọng của kiến trúc bộ nhớ cache hiện đại. Trace Cache đã được sử dụng để tăng tốc độ thực thi chương trình và giảm thời gian truy cập bộ nhớ chính. Nó đã được sử dụng trong các ứng dụng như máy tính để bàn, máy chủ, điện thoại thông minh và các thiết bị di động khác. Trong các bộ vi xử lý hiện đại, Trace Cache đã được kết hợp với các công nghệ khác như bộ nhớ cache cấp 2 và bộ nhớ cache cấp 3 để tăng hiệu suất và giảm thời gian truy cập bộ nhớ chính. Trace Cache cũng

đã được sử dụng trong các bộ vi xử lý đa nhân để tăng hiệu suất và tối ưu hóa sử dụng tài nguyên.

CHƯƠNG 3 THIẾT KẾ CỦA TRACE CACHE



Hình 3.1 Sơ đồ Trace Cache

Đây một sơ đồ khối của Trace Cache, một cấu trúc bộ nhớ đệm đặc biệt được sử dụng trong kiến trúc máy tính để cải thiện hiệu suất trong việc phân tích, dự đoán và thực thi các lệnh.

Cấu trúc của 1 Trace Cache bao gồm 2 phần: thông tin điều khiển (Control Information) và Vết chỉ dẫn (Instruction Trace).

Phần Control Information lại bao gồm Control Logic và địa chỉ của ô nhớ sẽ được tìm nạp tiếp theo hay còn gọi là Next Fetch Address.

Trong phần Control Logic, Trace Cache sẽ nhận địa chỉ tìm nạp và thông qua tag, branch flags, branch mask để biết được trace hit hay là trace miss. Nếu là trace miss thì Trace Cache này phải đợi. Còn là trace hit thì sẽ chuyển đến Fetch Address tiếp theo.

Điểm qua về khái niệm của 1 số thuật ngữ sử dụng trong Trace Cache:

- Tag ở đây có tác dụng xác định địa chỉ bắt đầu của vết, tức là các bit bậc cao hơn của địa chỉ cho chỉ dẫn đầu tiên của vết.

- Branch Flags là một bit duy nhất cho mỗi nhánh trong vết để cho biết có hay không nhánh đã được thực hiện hoặc không được thực hiện.

- Branch Mask: Cần có một số trạng thái để chỉ ra số lượng nhánh trong dấu vết và điều này cần Branch Mask quyết định. Thông tin này được cả bộ đệm theo dõi sử dụng để xác định có bao nhiêu cờ nhánh cần kiểm tra và bởi bộ dự đoán nhánh để cập nhật.

- Đến với phần Fetch Address tiếp theo thì ta có Target Address và Fall-through Address.

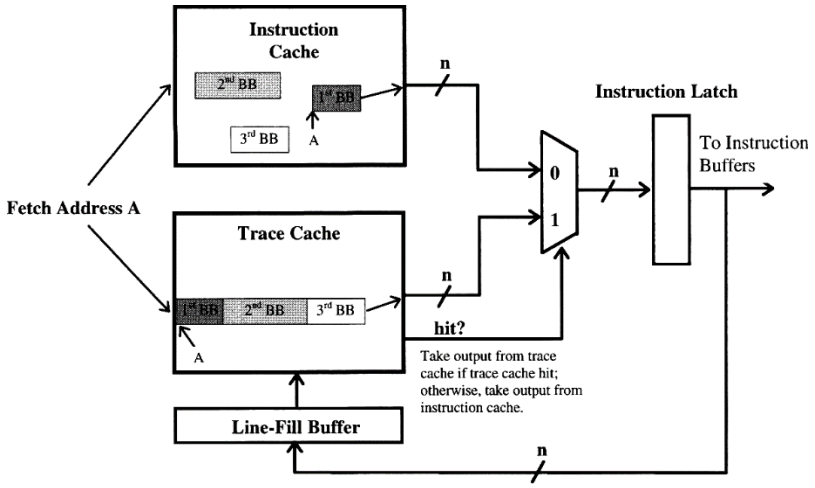
- Target Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng là một nhánh và đượxc dự đoán là đã thực hiện.

- Fall-through Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng của vết là một nhánh và được dự đoán là không được thực hiện.

Instruction cache được tạo ra bởi Line-Fill Buffer / Logic hay còn gọi là bộ đệm điền dòng hoặc logic điền dòng.

Bộ đệm điền dòng được sử dụng để chốt các khối cơ bản lại với nhau để tạo thành các vết được ghi vào Trace Cache. Các khối cơ bản được chốt từng cái một vào bộ đệm điền khi tiến hành tìm nạp lệnh. Logic điền (Fill logic) sẽ hợp nhất từng khối cơ bản mới với các hướng dẫn đã có trong bộ đệm điền vào dòng. Quá trình điền hoàn tất khi đạt được độ dài vết tối đa n hoặc số lượng nhánh tối đa trong vết m.

CHƯƠNG 4 CƠ CHẾ CỦA TRACE CACHE (CƠ CHẾ TÌM NẠP-THE FETCH MECHANISM)



Hình 4.1. Sơ đồ cơ chế tìm nạp ở Trace Cache

Fetch mechanism hay còn gọi là cơ chế tìm nạp, là một kỹ thuật tối ưu hóa bộ nhớ cache trên bộ xử lý để cải thiện hiệu suất của hệ thống. Đây là một kỹ thuật đặc biệt được sử dụng trong bộ xử lý của các máy tính có kiến trúc x86.

Sơ đồ tìm nạp sử dụng một Trace Cache hoạt động cùng với Standard instruction cache (Bộ đệm chỉ dẫn tiêu chuẩn). Vào đầu mỗi chu kỳ, Trace Cache được truy cập song song với trace instruction dựa trên địa chỉ tìm nạp. Công cụ dự đoán nhiều nhánh (multiple branch predictor) tạo ra các kết quả nhánh cho một vài nhánh tiếp theo trong khi các bộ đệm đang được truy cập.

Nếu có một lần truy cập được vào Trace Cache (Trace Cache hit), thì vết sẽ được đọc từ Trace Cache vào bộ đệm phát hành lệnh (Instruction issue buffer). Một Trace Cache hit xảy ra khi địa chỉ tìm nạp khớp với thẻ địa chỉ của vết và dự đoán nhánh khớp với hướng nhánh của vết.

Nếu một trong hai điều kiện này không đúng, thì sẽ xảy ra lỗi Trace Cache (Trace Cache miss) và quá trình tìm nạp lệnh thông thường được tiến hành từ instruction cache.

Cơ chế tìm nạp của Trace Cache có nhiều lợi ích:

- Giảm thiểu số lượng truy cập vào bộ nhớ chính, giúp giảm độ trễ và tăng tốc độ thực thi lệnh.

- Tăng khả năng dự đoán nhánh (branch prediction) bằng cách lưu trữ các instruction đã được thực thi trước đó trong các đoạn mã có cấu trúc tương tự. Điều này giúp bộ vi xử lý có thể dự đoán các nhánh (branch) của các đoạn mã tương tự trong tương lai một cách chính xác hơn, giảm số lần nhảy nhánh (branch misprediction) và tăng tốc độ thực thi.

Tuy nhiên, Trace Cache cũng có một số hạn chế:

- Thứ nhất, nó yêu cầu một lượng bộ nhớ đáng kể để lưu trữ các chuỗi lệnh, do đó có thể ảnh hưởng đến kích thước của bộ nhớ cache và tăng chi phí sản xuất.

- Thứ hai, Trace Cache có thể không hiệu quả khi các chuỗi lệnh không liên tục hoặc khi các lệnh được thực thi theo một cách không đồng nhất.

CHƯƠNG 5 TRACE CACHE MISS VÀ HIT LOGIC

5.1 Giới thiệu

Trong kiến trúc CPU hiện đại, Trace Cache là một trong những thành phần quan trọng giúp tăng tốc độ thực thi các chương trình máy tính. Bởi vì vậy việc xử lý Trace Cache Miss và Hit Logic sẽ có ảnh hưởng lớn đến hiệu suất của hệ thống. Nên trong phần này, chúng ta sẽ tìm hiểu về logic hit và miss của Trace Cache. Về cơ bản thì logic hit và miss của Trace Cache cũng tương tự như trên cache.

5.2 Cache hit

Là trạng thái dữ liệu được yêu cầu, được xử lý bởi một thành phần hoặc một ứng dụng được tìm thấy trong bộ nhớ cache

5.3 Cache miss

Là trạng thái nơi dữ liệu được yêu cầu xử lý bởi một thành phần hoặc ứng dụng không được tìm thấy trong bộ nhớ cache

→ Gây ra sự chậm trễ trong thực thi bởi vì nó phải đợi dữ liệu được yêu cầu từ bộ nhớ cache

5.4 Hit and miss logic ở Trace Cache

Các trace line được lưu trữ trong Trace Cache dựa trên PC của lệnh đầu tiên trong vết và một tập hợp các dự đoán rẽ nhánh. Điều này cho phép lưu trữ các trace line khác nhau bắt đầu trên cùng một địa chỉ, mỗi line đại diện cho các kết quả nhánh khác nhau.

Phương pháp gắn thẻ này giúp cung cấp khả năng kết hợp đường dẫn đến bộ nhớ cache theo dõi. Phương pháp khác có thể bao gồm việc chỉ khởi động PC dưới dạng thẻ trong Trace Cache.

Trong giai đoạn tìm nạp lệnh của một quy trình, PC hiện tại cùng với một tập hợp các dự đoán nhánh được kiểm tra trong bộ nhớ Trace Cache để tìm lần truy cập (hit).

Nếu có một lần hit, một trace line sẽ được cung cấp cho đơn vị tìm nạp mà không cần phải chuyển đến bộ đệm thông thường hoặc bộ nhớ cho các chỉ dẫn này. Trace Cache tiếp tục cung cấp nguồn cho đơn vị tìm nạp cho đến khi trace line kết thúc hoặc cho đến khi có dự đoán sai trong đường dẫn.

Nếu có sai sót (miss), một dấu vết mới sẽ bắt đầu được xây dựng.

5.5 Tác dụng

Tác dụng của cache hit và miss logic là cải thiện hiệu suất của hệ thống máy tính. Khi CPU cần truy cập dữ liệu hoặc hướng dẫn, nó trước tiên kiểm tra xem chúng đã được lưu trữ trong cache hay chưa.

Nếu dữ liệu hoặc hướng dẫn đã được lưu trữ trong cache, CPU có thể truy cập nhanh chóng và trực tiếp từ cache mà không cần phải truy cập đến bộ nhớ chính. Điều này được gọi là cache hit. Việc cache hit giúp cải thiện hiệu suất của hệ thống bởi vì nó giảm thời gian truy cập đến bộ nhớ chính và tăng tốc độ xử lý của CPU.

Ngược lại, nếu dữ liệu hoặc hướng dẫn chưa được lưu trữ trong cache, CPU phải truy cập đến bộ nhớ chính để lấy chúng. Điều này được gọi là cache miss. Việc cache miss sẽ làm giảm hiệu suất của hệ thống bởi vì nó tạo ra độ trễ trong việc truy cập dữ liệu hoặc hướng dẫn.

5.6 Cách tối ưu hóa hiệu năng thông qua Trace Cache hit và miss logic

Các nhà sản xuất CPU đã nghiên cứu và phát triển nhiều giải pháp để giảm thiểu lỗi Trace Cache Miss và tối ưu hoá Trace Cache Hit Logic. Một số giải pháp như sau:

Tăng kích thước của Trace Cache để cải thiện khả năng lưu trữ các trace của chương trình.

Sử dụng các kỹ thuật dự đoán nhánh để giảm thiểu Trace Cache Miss.

Tối ưu hoá cơ chế kiểm tra dữ liệu tại Trace Cache Hit để giảm thiểu lỗi cache và tăng tốc độ xử lý.

CHƯƠNG 6 MẶT LỢI VÀ MẶT HẠI CỦA TRACE CACHE

a. MẶT LỢI:

Trace Cache có nhiều mặt lợi ích, bao gồm:

1. Giảm thiểu số lượng truy cập vào bộ nhớ chính: Trace Cache giúp giảm thiểu số lượng truy cập vào bộ nhớ chính bởi vì nó lưu trữ các chuỗi lệnh đã được thực thi gần đây nhất và cung cấp chúng cho bộ phận tìm nạp thay vì yêu cầu bộ phận này phải tìm nạp từ bộ nhớ chính. Điều này giúp giảm độ trễ và tăng tốc độ thực thi lệnh.

2. Tăng hiệu suất của các vi xử lý Superscalar hoặc VLIW: Trace Cache cung cấp các chuỗi lệnh liên tục để thực thi đồng thời, thay vì chỉ thực thi từng lệnh một. Điều này giúp tăng hiệu suất của các vi xử lý Superscalar hoặc VLIW bằng cách tận dụng khả năng thực thi đồng thời của các lệnh.

3. Giảm độ trễ: Trace Cache giúp giảm độ trễ bởi vì nó cung cấp các chuỗi lệnh liên tục để thực thi đồng thời, thay vì chỉ thực thi từng lệnh một. Điều này giúp giảm thời gian chờ đợi giữa các lệnh và tăng tốc độ thực thi lệnh.

4. Tăng khả năng dự đoán nhánh: Trace Cache có thể giúp tăng khả năng dự đoán nhánh bởi vì nó có thể lưu trữ các chuỗi lệnh liên tục trong các khối lệnh thực thi, bao gồm cả các lệnh nhánh. Điều này giúp tăng khả năng dự đoán nhánh và giảm số lần phải chờ đợi khi dự đoán nhánh sai.

5. Giảm chi phí sản xuất: Trace Cache có thể giúp giảm chi phí sản xuất bởi vì nó giúp giảm kích thước của bộ nhớ cache và giảm số lượng truy cập vào bộ nhớ chính. Điều này giúp giảm chi phí sản xuất và tăng hiệu quả của việc sử dụng bộ nhớ cache.

6. Tăng hiệu suất của các ứng dụng đa luồng: Trace Cache có thể tăng hiệu suất của các ứng dụng đa luồng bởi vì nó cung cấp các chuỗi lệnh liên tục để thực thi đồng thời, giúp tận dụng tối đa khả năng đa luồng của bộ xử lý.

7. Giảm độ trễ khi thực thi lệnh nhảy: Trace Cache có thể giảm độ trễ khi thực thi lệnh nhảy bởi vì nó có thể lưu trữ các chuỗi lệnh liên tục trong các khối lệnh thực thi, bao gồm cả các lệnh nhảy. Điều này giúp giảm số lần phải chờ đợi khi thực thi lệnh nhảy và tăng tốc độ thực thi lệnh.

8. Tăng hiệu suất của các ứng dụng có tính chất lặp lại cao: Trace Cache có thể tăng hiệu suất của các ứng dụng có tính chất lặp lại cao bởi vì nó có thể lưu trữ các chuỗi lệnh liên tục trong các khối lệnh thực thi, bao gồm cả các lệnh lặp lại. Điều này giúp tận dụng tối đa tính chất lặp lại của các ứng dụng và tăng tốc độ thực thi lệnh.

9. Tăng khả năng tùy chỉnh: Trace Cache có thể được tùy chỉnh để phù hợp với các ứng dụng cụ thể bằng cách điều chỉnh kích thước của cache hoặc cách thức lưu trữ các chuỗi lệnh. Điều này giúp tối ưu hóa hiệu suất của Trace Cache cho các ứng dụng cụ thể.

Tóm lại, Trace Cache là một công nghệ quan trọng trong vi xử lý hiện đại, giúp tăng hiệu suất và giảm độ trễ của bộ xử lý. Nó có nhiều lợi ích và có thể được tùy chỉnh để phù hợp với các ứng dụng cụ thể.

b. <u>MẶT HẠI:</u>

Trace Cache là một loại bộ nhớ cache được sử dụng trong các vi xử lý để cải thiện hiệu suất của chúng. Tuy nhiên, như mọi công nghệ khác, Trace Cache cũng có những mặt hại của nó. Dưới đây là tổng hợp chi tiết các mặt hại của Trace Cache:

Trace Cache yêu cầu sử dụng nhiều bộ nhớ cache để lưu giữ các thông tin về các cấu trúc lặp lại của chương trình. Điều này có thể dẫn đến sự lãng phí tài nguyên bộ nhớ và tăng độ phức tạp của hệ thống → Sử dụng nhiều bộ nhớ cache

Trace Cache hoạt động tốt với các chương trình có cấu trúc lặp lại, nhưng không hiệu quả đối với các chương trình không có cấu trúc lặp lại. Điều này có thể dẫn đến tình trạng lãng phí tài nguyên và không cải thiện được hiệu suất của hệ thống → Không hiệu quả đối với các chương trình không có cấu trúc lặp lại

Trong một số trường hợp, Trace Cache có thể dẫn đến thời gian truy xuất chậm hơn khi so sánh với bộ nhớ cache thông thường. Điều này có thể xảy ra khi chương trình không có cấu trúc lặp lại hoặc khi các mẫu lặp lại không phù hợp với các cấu trúc dữ liệu của Trace Cache → Thời gian truy xuất chậm

Trace Cache là một công nghệ phức tạp và khó điều chỉnh. Việc tối ưu Trace Cache để cải thiện hiệu suất của hệ thống có thể đòi hỏi sự hiểu biết chuyên sâu về kiến trúc máy tính và lập trình → Khó điều chỉnh và tối ưu

Việc thiết kế và triển khai Trace Cache có thể làm tăng độ phức tạp của hệ thống và yêu cầu nhiều kiến thức chuyên môn để vận hành hiệu quả → Tăng độ phức tạp của hệ thống

Do tính chất phức tạp của Trace Cache, việc tối ưu hoá nó để đạt được hiệu suất cao nhất có thể là một thách thức lớn → Khó khăn trong việc tối ưu hoá

=> Nhìn chung, mặc dù Trace Cache có nhiều lợi ích trong việc cải thiện hiệu suất của bộ xử lý, nhưng cũng có một số mặt hạn chế cần được xem xét khi sử dụng. Việc lựa chọn sử dụng Trace Cache hay không phụ thuộc vào nhiều yếu tố khác nhau và cần được cân nhắc kỹ lưỡng.

TÀI LIỆU THAM KHẢO

[1] Ho-Seop Kim and James E. Smith, *Depart of Electrical and Computer Engineering University of Wisconsin—Madison*, Submitted to the 30t International Symposium on Computer Architecture (ISCA 2003)

[2] Software Trace Cache, Computer Architecture Department Universitat Politkcnica de Catalunya Jordi Girona 1-3, Module D6 08034 Barcelona (Spain)

[3] Eric Rotenberg Computer Science Dept. Univ. of Wisconsin – Madison, Steve Bennett Intel Corporation, James E. Smith Dept. of Elec. and Comp. Engr. Univ. of Wisconsin – Madison, *Trace Cache: a Low Latency Approach to High Bandwidth Instruction Fetching*

[4] Sanjay Jeram Patel, Daniel Holmes Friendly, and Yale N. Patt, *Advanced Computer Architecture Laboratory Department of Electrical Engineering and Computer Science The University of Michigan, Critical Issues Regarding the Trace Cache Fetch Mechanism*

[5] Ravi Bhargava and Lizy K. John, *Technical Report TR-033103-01 Laboratory for Computer Architecture The University of Texas at Austin, Cluster Assignment Strategies for a Clustered Trace Cache Processor*, March 31, 2003

[6] Daksh S. Dharod, *Graduate Assignment, ECE 585, Portland State University, Trace Cache*